
ChronoHive Eval API — Reference

API reference

CONFIDENTIAL — PROPRIETARY

**Licensed solely for evaluation under the ChronoHive Terms
of Confidentiality and the ChronoHive Evaluation License.**

Do not distribute, copy, or publicly host.

© 2026 Layer1Labs Silicon Inc. All rights reserved.

The complete reference for the hosted ChronoHive evaluation API. This package is a client package: it drives this API; there is no server to self-host in this repository.

Honesty appendix (applies to every endpoint): what you are testing is **real** — this is a live hosted API, and every grant/refuse decision is produced by the real **ChronoHive Runtime kernel** (the actual admission-discipline logic: capacity accounting, plan validation, dependency checks — not a mock or a lookup table). What is **simulated** is the environment around the kernel: the storage backend, the incast contention model, and the storage API surface. No evaluation package can ship a data center, so the kernel's real decisions play out against a modeled world. All numbers are therefore simulation outcomes — valid for comparing coordination behavior — not hardware measurements. Every response carries `"simulated_backend": true` to say so.

Base URL

All paths below are relative to the hosted base URL:

```
https://api.layer1labs.ai
```

The deployment's version is reported by `GET /v1/health (version)` and in the `Server` header. Additive endpoint changes bump the minor version; breaking changes bump the major version.

Authentication

Every endpoint except `GET /v1/health`, `GET /v1/toc`, and `GET /` requires an API key:

```
Authorization: Bearer <raw-key>
```

Keys are issued by the operator, have an expiry date, and are stored server-side as salted hashes — the server never sees your key except in the Authorization header.

Failure	Status	Meaning
Missing/malformed header	401	No Bearer credentials sent
Unknown or expired key	401	Key not recognized or past expires_at

Executing the TOC (required before use)

Eval endpoints stay locked until your API key has a verified Terms of Confidentiality acceptance on file. Without it you get:

```
{
  "error": "toc_acceptance_required",
  "message": "This API key has not executed the Terms of Confidentiality...",
  "toc_sha256": "<pinned hash>",
  "accept_endpoint": "/v1/toc/accept",
  "toc_endpoint": "/v1/toc"
}
```

Step 1 — read the canonical TOC. Fetch it from the server so you sign exactly the text it enforces:

```
curl https://api.layer1labs.ai/v1/toc
```

Step 2 — sign it. The signing tool generates an ed25519 keypair (pure Python, no dependencies), binds the TOC hash to your key id and identity, and produces an acceptance token:

```
python3 tools/sign_toc.py --api-url https://api.layer1labs.ai \
  --key-id <your-key-id> \
  --name "Jane Doe" --org "Example Corp" --email jane@example.com
```

This writes `toc_signing.key` (mode 600 — your proof of execution, keep it, never commit it) and `toc_acceptance.json`.

Step 3 — submit the token:

```
curl -X POST https://api.layer1labs.ai/v1/toc/accept \
  -H "Authorization: Bearer <raw-key>" \
  -H "Content-Type: application/json" \
  -d @toc_acceptance.json
```

Or do all three steps at once with `--submit`:

```
python3 tools/sign_toc.py --api-url https://api.layer1labs.ai \
  --api-key <raw-key> --key-id <your-key-id> --submit
```

A 200 `{"accepted": true, ...}` means your key is activated. Per TOC §5, a signature verified against the pinned TOC hash constitutes execution of the TOC with the same force as a handwritten signature.

Endpoints

GET /v1/health

Liveness probe. Unauthenticated.

```
curl https://api.layer1labs.ai/v1/health
```

```
{
  "status": "ok",
  "version": "1.1.0",
  "toc_sha256": "<pinned TOC hash>",
  "simulated_backend": true
}
```

GET /v1/toc

The canonical Terms of Confidentiality text and its SHA-256 digest. Unauthenticated. Sign exactly this text.

Response:

```
{
  "toc_sha256": "<hex>",
  "toc_text": "<full TOC.md text>",
  "accept_endpoint": "/v1/toc/accept",
  "simulated_backend": true
}
```

POST /v1/toc/accept

Submit a signed acceptance token (see "Executing the TOC"). Authenticated — the token's `key_id` must match the calling key.

Request body: the acceptance token object:

```
{
  "protocol": "CHRONOHIVE-TOC-ACCEPT/v1",
  "toc_sha256": "<hex>",
  "key_id": "<your-key-id>",
  "signer_name": "Jane Doe",
  "organization": "Example Corp",
  "email": "jane@example.com",
  "timestamp": 1790000000,
  "public_key": "<hex ed25519 public key>",
  "signature": "<hex ed25519 signature>"
}
```

Success (200):

```
{
  "accepted": true,
  "key_id": "<your-key-id>",
  "toc_sha256": "<hex>",
  "signer": "Jane Doe",
  "organization": "Example Corp",
  "accepted_at": 1790000000,
  "simulated_backend": true
}
```

Failures: 401 (bad key), 403 {"error": "toc_acceptance_rejected", "reason": "..."} — hash mismatch, key_id mismatch, bad signature, timestamp in the future (beyond 10 minutes of clock skew), or implausibly old timestamp (before 2020-01-01).

accepted_at is the server's own clock at accept time — the authoritative execution record — never the client-supplied token timestamp. The acceptance survives restarts.

POST /v1/scenarios

Run the deterministic checkpoint/prefetch admission simulation. Authenticated + TOC required. Quota: **quota_scenarios_per_day** per key (default 50/day).

Request:

```
{
  "jobs": 8,
  "ckpt_gb": 200.0,
  "prefetch_gb": 150.0,
  "incast_alpha": 0.2,
  "seeds": [7, 21],
  "miss_penalty_s": 60.0
}
```

Field	Type	Default	Bounds	Meaning
jobs	int	8	1-64	Simulated training jobs
ckpt_gb	float	200.0	1-10000	Checkpoint size per job (GB)
prefetch_gb	float	150.0	1-10000	Prefetch size per job (GB)
incast_alpha	float	0.2	0-1	Contention severity

seeds	int[]	[7]	1-10 seeds	Deterministic seeds; same seed $\Rightarrow$ same result
miss_penalty_s	float	60.0	0-3600	GPU stall seconds per checkpoint miss

Response (200): a `scenario_id` plus the full result. Per seed, the result compares **baseline** (no admission control) against **admission** (every transfer arbitrated by the kernel):

```
{
  "scenario_id": "a3f9c1...",
  "simulated_backend": true,
  "result": {
    "simulated_backend": true,
    "honesty": "Storage backend, contention model, ...",
    "params": { "jobs": 8, "ckpt_gb": 200.0, "...": "..." },
    "seeds": [
      {
        "seed": 7,
        "baseline": { "gpu_stall_s": 1234.0, "ckpt_misses": 5,
                     "makespan_s": 9000.0, "jain_fairness": 0.97 },
        "admission": { "gpu_stall_s": 12.0, "ckpt_misses": 0,
                      "makespan_s": 8990.0, "jain_fairness": 0.99,
                      "kernel_admits": 42, "kernel_refusals": 3 },
        "verdict": { "stall_reduction": 0.99,
                     "makespan_delta_s": -10.0, "pass": true }
      }
    ]
  }
}
```

The verdict passes on $\geq 20\%$ **GPU-stall reduction with no makespan regression** (tolerance 1%). `kernel_admits` / `kernel_refusals` are the real kernel's per-seed counters.

GET /v1/scenarios/{id}

Fetch a previously run scenario result. Authenticated + TOC required. 404 for an unknown id.

POST /v1/admission/decide

The live kernel decision path: submit transfer requests for one scheduling window and get grant/refuse decisions back. Authenticated + TOC required. Quota: `quota_decide_per_min` per key (default 60/min). This is the endpoint a storage controller would call per scheduling window.

Request:

```
{
  "now_s": 0,
  "window": 30,
  "requests": [
    { "dataset_id": "job0.ckpt", "job_id": 0, "kind": "checkpoint",
      "bytes_remaining": 200000000000, "deadline_s": 120 },
    { "dataset_id": "job2.prefetch", "job_id": 2, "kind": "prefetch",
      "bytes_remaining": 150000000000 }
  ]
}
```

Field	Type	Bounds	Meaning
<code>now_s</code>	float	≥ 0	Window start (s)
<code>window</code>	int	≥ 0	Window index (opaque to the kernel)
<code>requests</code>	object[]	1-256	Transfer requests
↳ <code>dataset_id</code>	string	required	Dataset identifier
↳ <code>job_id</code>	int	default 0	Owning job
↳ <code>kind</code>	string	checkpoint prefetch \	Transfer kind
↳ <code>bytes_remaining</code>	float	> 0	Bytes still to move
↳ <code>deadline_s</code>	float \	null	$> \text{now_s}$ Deadline; null = best effort

Response (200):

```
{
  "simulated_backend": true,
  "honesty": "Storage backend, contention model, ...",
  "window": 30,
  "admitted": { "job0.ckpt": 5000000000.0 },
  "refused": ["job2.prefetch"],
  "qos": { "job0.ckpt": 63 },
  "kernel": { "admits": 1, "refusals": 1 }
}
```

`admitted` maps dataset → granted bandwidth (B/s). `refused` lists datasets the kernel declined this window (retry them in a later window). `qos` maps dataset → QoS level 0–63, driven by the grant decisions on the simulated storage surface. `kernel` carries the kernel's admit/refuse counters.

Admission policy: earliest-deadline-first against the provisioned write/read pipes; best-effort (deadline-less) requests yield to deadlined ones.

POST /v1/admission/compare

Kernel-real coordination comparison: the preregistered LF workload runs twice through the real kernel — an uncoordinated baseline and the coordinated run — and the API returns the coordination delta, honestly labeled. Authenticated + TOC required. Takes no request body beyond {}.

Response (200) carries both runs' kernel counters and the measured delta (stall, misses, admits/refusals), each stamped `"simulated_backend": true`.

POST /v1/lf/compile

Compile a **complete Lingua Franca project** to a `.chb` blob through the pinned `lfc + chro` toolchain. Authenticated + TOC required, and the key must hold a live **compile license** (issued by the operator — without one the endpoint returns 403 `compile_license_required`).

The request carries the whole project as a file map:

```
{
  "files": {
    "IoCoordinator.lf": "<LF source>",
    "lib/Storage.lf": "<LF source>"
  },
  "entrypoint": "IoCoordinator.lf",
  "params": { "steps": 200 },
}
```

```
"capacities": { "storage_bw": 100 }  
}
```

Field	Type	Required	Meaning
files	object	yes	Relative path → LF source for every file in the project
entrypoint	string	yes	Which file is the compile entrypoint (must be a key of files)
params	object	no	Integer main-reactor parameter overrides (e.g. steps)
capacities	object	no	Non-negative-integer resource capacities (e.g. storage_bw)

Enforced limits (the server rejects anything over):

Limit	Value
Files per project	64
Per-file size	512 KiB
Aggregate project size	2 MiB
Path length	256 chars; clean relative .lf paths only
Compile timeout	120 s

Response (200):

```
{  
  "blob_base64": "<base64 CHB1 blob>",  
  "blob_sha256": "<hex>",  
  "input_sha256": "<hex, deterministic hash of the project + params + capacities>",  
  "input_files": ["IoCoordinator.lf"],  
  "entrypoint": "IoCoordinator.lf",  
  "lfc_version": "lfc 0.13.0",  
}
```

```

"chronoc_version": "0.1.0",
"ops": 3,
"steps": 200,
"simulated_backend": true
}

```

`input_sha256` is deterministic: the same project, params, and capacities always hash the same, so you can cache blobs keyed on it. The server validates the project with the pinned `lfc` v0.13.0 before `chronoc` lowers it — anything `lfc` rejects is never compiled.

If the deployment's compiler toolchain is not provisioned, the endpoint returns `503 compiler_unavailable`. The client in this package reports the server's answer verbatim in that case.

The canonical client for this endpoint is `clients/compile_client.py`:

```

python3 clients/compile_client.py --api-url https://api.layer1labs.ai \
  --api-key "$CHRONOHIVE_API_KEY" --capacity storage_bw=100 -o /tmp/io.chb

```

It builds the file map from a project directory (`lf/` by default), validates the contract locally before sending, and verifies the returned blob's structure, provenance, and `blob_sha256` on receipt.

GET /

Minimal eval console (HTML): the TOC step plus a scenario form. Unauthenticated. Useful for a first look; scripted evaluation should use the JSON endpoints.

Quotas and rate limits

Quotas are per API key and configured at issuance:

Quota	Default	Window	On exceed
<code>quota_scenarios_per_day</code>	50	rolling 24 h	429, Retry-After header
<code>quota_decide_per_min</code>	60	rolling 60 s	429, Retry-After header

429 bodies carry `retry_after_s`. Quota counters are persistent.

Audit

Every authenticated call is appended to a JSONL audit log with the key id, method, path,

status, request-body SHA-256, and duration. Raw keys and request bodies are never logged.

Error catalog

Status	error	Meaning
400	(various)	Invalid parameters; the message says which
401	missing or malformed Authorization header	No/invalid credentials format
401	invalid or expired API key	Unknown key or past expiry
403	toc_acceptance_required	Key valid, TOC not yet executed
403	toc_acceptance_rejected	Token failed verification (reason explains)
403	compile_license_required	Key valid + TOC executed, but no live compile license
404	not found / unknown scenario id	Bad path or id
405	use GET /v1/toc	Wrong method on the TOC document endpoint
429	scenario quota exceeded / decide quota exceeded	Slow down; honor Retry-After
503	compiler_unavailable	Compile requested but the toolchain isn't provisioned on this deployment

Versioning

Version	Changes
---------	---------

current server release	/v1/lf/compile (complete-project file map, license-gated), /v1/admission/compare (kernel-real coordination comparison), per-request usage logging
------------------------	---

1.1.0	TOC execution gate (/v1/toc, /v1/toc/accept, 403 gating)
-------	--

1.0.0	Initial eval API
-------	------------------

Check `GET /v1/health` for the version your deployment serves.